Material do Estudante — Desafio 7 — Sequência de graus

Estrutura de Dados II | Unidade III — Grafos | Profa. Kadidja Valéria

Nome: Gabriel Cristian da Cunha Valentim	**Turma:** N1	**Data:** 09/10/2026

Construir um grafo simples com oito vértices e sequência de graus, em ordem não decrescente, **(1, 1, 2, 3, 3, 4, 4, 6)**.

Análise prévia

Soma dos graus pedidos = 1 + 1 + 2 + 3 + 3 + 4 + 4 + 6 = **24**, que é par; pelo Teorema do Aperto de Mãos, o grafo deve ter 24 ÷ 2 = **12 arestas**. O maior grau é 6 ≤ n − 1 = 7, então é possível em um grafo simples (um vértice pode ser vizinho de no máximo os outros 7).

1. Vértices

V = {A, B, C, D, E, F, G, H}. Distribuí os graus do maior para o menor: A = 6, B = 4, C = 4, D = 3, E = 3, F = 2, G = 1, H = 1.

2. Construção começando pelo vértice de maior grau

Usei o procedimento de Havel-Hakimi: o vértice que ainda precisa de mais arestas é ligado aos que também precisam de mais; cada vizinho escolhido perde 1 no grau que falta. Assim nunca há laço nem aresta repetida.

Passo	Vértice	Liga com	Graus que ainda faltam (A B C D E F G H)
1	A (6)	B, C, D, E, F, G	0 3 3 2 2 1 0 1
2	B (3)	C, D, E	0 0 2 1 1 1 0 1
3	C (2)	D, E	0 0 0 0 0 1 0 1
4	F (1)	H	0 0 0 0 0 0 0 0

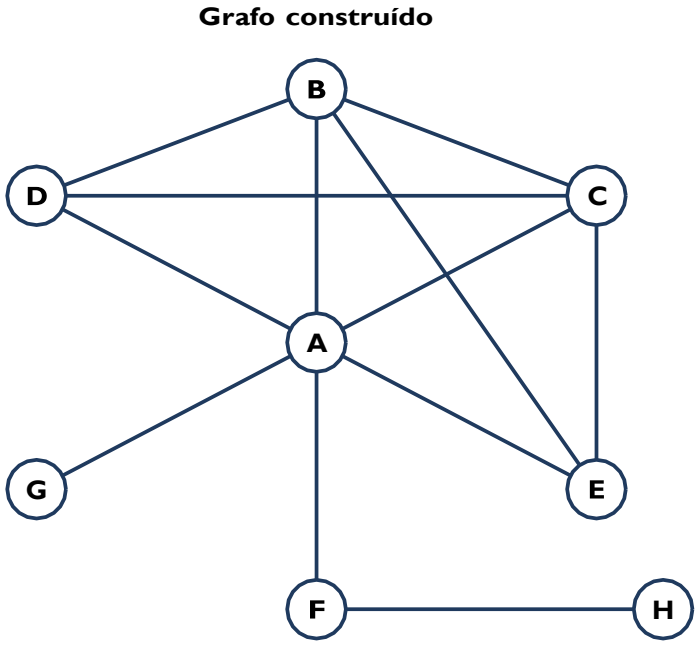


Figura 1 — Grafo simples obtido (12 arestas, sem laços e sem arestas paralelas).

3. Conjunto de arestas E

E = {(A,B), (A,C), (A,D), (A,E), (A,F), (A,G), (B,C), (B,D), (B,E), (C,D), (C,E), (F,H)}

4. Grau de cada vértice

Vértice	Vizinhos	Grau obtido
A	B, C, D, E, F, G	6
B	A, C, D, E	4
C	A, B, D, E	4
D	A, B, C	3
E	A, B, C	3
F	A, H	2
G	A	1
H	F	1

Sequência obtida (ordenada): **(1, 1, 2, 3, 3, 4, 4, 6)** — igual à pedida.

5. Verificação

Soma dos graus	6 + 4 + 4 + 3 + 3 + 2 + 1 + 1 = 24
Número de arestas	12
Soma dos graus = 2 × número de arestas?	Sim: 24 = 2 × 12
Grafo simples?	Sim: nenhuma aresta (X,X) e nenhum par repetido em E

Justificativa: cada aresta tem duas extremidades e contribui com 1 para o grau de cada uma, logo a soma dos graus é sempre o dobro do número de arestas. A igualdade 24 = 2 × 12 confirma a contagem, e a sequência recalculada a partir de E coincide com a solicitada. O programa em Java faz a construção e repete todas essas conferências automaticamente.

Implementação em Java

Plataforma utilizada: **Java** (OpenJDK 21, compilado com `--release 8`, portanto roda em qualquer JVM a partir do Java 8). Arquivos entregues: `Desafio7SequenciaGraus.java` (código-fonte) e `Desafio7SequenciaGraus.class` (bytecode compilado).

Para executar: `java Desafio7SequenciaGraus` na pasta do arquivo .class (ou recompilar com `javac Desafio7SequenciaGraus.java`).

Saída do programa (execução real):

```
=== DESAFIO 7 - SEQUENCIA DE GRAUS ===
Sequencia pedida (nao decrescente): (1, 1, 2, 3, 3, 4, 4, 6)
Grau desejado de cada vertice: A=6, B=4, C=4, D=3, E=3, F=2, G=1, H=1
Soma dos graus pedidos = 24 (par -> 12 arestas)
Maior grau = 6 <= n - 1 = 7? sim

=== Construcao passo a passo (comecando pelo maior grau) ===
Passo 1: A precisa de 6 aresta(s) -> liga com B, C, D, E, F, G
         graus que ainda faltam: A=0 B=3 C=3 D=2 E=2 F=1 G=0 H=1
Passo 2: B precisa de 3 aresta(s) -> liga com C, D, E
         graus que ainda faltam: A=0 B=0 C=2 D=1 E=1 F=1 G=0 H=1
Passo 3: C precisa de 2 aresta(s) -> liga com D, E
         graus que ainda faltam: A=0 B=0 C=0 D=0 E=0 F=1 G=0 H=1
Passo 4: F precisa de 1 aresta(s) -> liga com H
         graus que ainda faltam: A=0 B=0 C=0 D=0 E=0 F=0 G=0 H=0

=== Conjunto de arestas E ===
E = {(A,B), (A,C), (A,D), (A,E), (A,F), (A,G), (B,C), (B,D), (B,E), (C,D), (C,E), (F,H)}

=== Grau obtido de cada vertice ===
  A | grau 6 | vizinhos: B C D E F G
  B | grau 4 | vizinhos: A C D E
  C | grau 4 | vizinhos: A B D E
  D | grau 3 | vizinhos: A B C
  E | grau 3 | vizinhos: A B C
  F | grau 2 | vizinhos: A H
  G | grau 1 | vizinhos: A
  H | grau 1 | vizinhos: F

=== Verificacao ===
Sequencia obtida: (1, 1, 2, 3, 3, 4, 4, 6)
Igual a pedida? sim
Soma dos graus = 24
Numero de arestas = 12
Soma dos graus = 2 x numero de arestas? 24 = 2 x 12 = 24 -> sim
Grafo simples (sem lacos e sem arestas paralelas)? sim
```

Código-fonte: Desafio7SequenciaGraus.java

```
/*
 * Estrutura de Dados II - Unidade III: Grafos
 * Profa. Kadidja Valeria
 *
 * Desafio 7 - Sequencia de graus
 * Aluno: Arthur Santos Aguiar
 * Turma: N1
 * Data: 09/10/2026
 *
 * Objetivo: construir um grafo SIMPLES com 8 vertices e sequencia de graus
 * (1, 1, 2, 3, 3, 4, 4, 6).
 *
 * Estrategia (algoritmo de Havel-Hakimi):
 *   - Cada vertice recebe o grau que ainda precisa alcancar ("resto").
 *   - A cada passo, pega o vertice com MAIOR resto e liga ele aos d vertices
 *     seguintes de maior resto (d = resto dele). Esses vertices perdem 1.
 *   - Repete ate todos os restos ficarem zero.
 *   Como cada par so e ligado uma vez e nunca se liga um vertice a ele mesmo,
 *   nao surgem lacos nem arestas paralelas.
 * No final o programa recalcula os graus a partir das arestas e confere
 * a soma dos graus com 2 x |E|.
 *
 * Compilar: javac Desafio7SequenciaGraus.java
 * Executar: java Desafio7SequenciaGraus
 */
import java.util.ArrayList;
import java.util.Arrays;
import java.util.List;

public class Desafio7SequenciaGraus {

    public static void main(String[] args) {
        int[] pedida = {1, 1, 2, 3, 3, 4, 4, 6};
        String[] v = {"A", "B", "C", "D", "E", "F", "G", "H"};
        int n = v.length;

        titulo("DESAFIO 7 - SEQUENCIA DE GRAUS");
        System.out.println("Sequencia pedida (nao decrescente): " + formatar(pedida));

        // A recebe o maior grau, B o seguinte, ..., H o menor
        int[] alvo = new int[n];
        for (int i = 0; i < n; i++) {
            alvo[i] = pedida[n - 1 - i];
        }
        System.out.print("Grau desejado de cada vertice: ");
        for (int i = 0; i < n; i++) {
            System.out.print(v[i] + "=" + alvo[i] + (i < n - 1 ? ", " : "\n"));
        }

        // testes rapidos antes de construir
        int soma = 0;
        for (int d : pedida) {
            soma += d;
        }
        System.out.println("Soma dos graus pedidos = " + soma
                + (soma % 2 == 0 ? " (par -> " + soma / 2 + " arestas)" : " (impar -> impossivel)"));
        System.out.println("Maior grau = " + pedida[n - 1] + " <= n - 1 = " + (n - 1) + "? "
                + simNao(pedida[n - 1] <= n - 1));
        if (soma % 2 != 0 || pedida[n - 1] > n - 1) {
            System.out.println("Sequencia nao pode ser de grafo simples.");
            return;
        }

        // ---------------- construcao ----------------
        titulo("Construcao passo a passo (comecando pelo maior grau)");
        boolean[][] adj = new boolean[n][n];
        List<String> arestas = new ArrayList<String>();
        int[] resto = alvo.clone();
        int passo = 1;

        while (true) {
            int u = maiorResto(resto);
            if (resto[u] == 0) {
                break; // todos ja atingiram o grau desejado
            }
            int d = resto[u];
            Integer[] candidatos = ordenarPorResto(resto, u);
```

```
            StringBuilder sb = new StringBuilder();
            sb.append("Passo ").append(passo).append(": ").append(v[u])
              .append(" precisa de ").append(d).append(" aresta(s) -> liga com ");
            for (int t = 0; t < d; t++) {
                int w = candidatos[t];
                if (resto[w] == 0 || adj[u][w]) {
                    System.out.println("Nao foi possivel completar: a sequencia nao e grafica.");
                    return;
                }
                adj[u][w] = true;
                adj[w][u] = true;
                resto[w]--;
                arestas.add("(" + v[u] + "," + v[w] + ")");
                sb.append(v[w]).append(t < d - 1 ? ", " : "");
            }
            resto[u] = 0;
            System.out.println(sb);
            System.out.print("     graus que ainda faltam: ");
            for (int i = 0; i < n; i++) {
                System.out.print(v[i] + "=" + resto[i] + (i < n - 1 ? " " : "\n"));
            }
            passo++;
        }

        // ---------------- registro ----------------
        titulo("Conjunto de arestas E");
        System.out.println("E = {" + String.join(", ", arestas) + "}");

        titulo("Grau obtido de cada vertice");
        int[] obtido = new int[n];
        for (int i = 0; i < n; i++) {
            StringBuilder viz = new StringBuilder();
            for (int j = 0; j < n; j++) {
                if (adj[i][j]) {
                    obtido[i]++;
                    viz.append(v[j]).append(" ");
                }
            }
            System.out.println("  " + v[i] + " | grau " + obtido[i] + " | vizinhos: " + viz.toString().trim());
        }

        // ---------------- verificacao ----------------
        titulo("Verificacao");
        int[] ordenada = obtido.clone();
        Arrays.sort(ordenada);
        System.out.println("Sequencia obtida: " + formatar(ordenada));
        System.out.println("Igual a pedida? " + simNao(Arrays.equals(ordenada, pedida)));

        int somaObtida = 0;
        for (int d : obtido) {
            somaObtida += d;
        }
        int m = arestas.size();
        System.out.println("Soma dos graus = " + somaObtida);
        System.out.println("Numero de arestas = " + m);
        System.out.println("Soma dos graus = 2 x numero de arestas? " + somaObtida + " = 2 x " + m
                + " = " + (2 * m) + " -> " + simNao(somaObtida == 2 * m));

        boolean temLaco = false;
        for (int i = 0; i < n; i++) {
            temLaco = temLaco || adj[i][i];
        }
        // como a matriz e booleana, um mesmo par nao pode ter duas arestas
        System.out.println("Grafo simples (sem lacos e sem arestas paralelas)? " + simNao(!temLaco));
    }

    // indice do vertice com maior resto (em empate, o de menor indice)
    static int maiorResto(int[] resto) {
        int u = 0;
        for (int i = 1; i < resto.length; i++) {
            if (resto[i] > resto[u]) {
                u = i;
            }
        }
        return u;
    }

    // demais vertices ordenados por resto decrescente (sort estavel: empate mantem ordem alfabetica)
    static Integer[] ordenarPorResto(final int[] resto, int excluir) {
        Integer[] idx = new Integer[resto.length - 1];
```

```
        int k = 0;
        for (int i = 0; i < resto.length; i++) {
            if (i != excluir) {
                idx[k++] = i;
            }
        }
        Arrays.sort(idx, (x, y) -> resto[y] - resto[x]);
        return idx;
    }

    static String formatar(int[] s) {
        StringBuilder sb = new StringBuilder("(");
        for (int i = 0; i < s.length; i++) {
            sb.append(s[i]).append(i < s.length - 1 ? ", " : "");
        }
        return sb.append(")").toString();
    }

    static String simNao(boolean b) {
        return b ? "sim" : "nao";
    }

    static void titulo(String t) {
        System.out.println();
        System.out.println("=== " + t + " ===");
    }
}
```